

# Transfer Learning / Pretraining

## Contents

- Transfer Learning / Pretraining
- Import packages
- Change data inputs here
- Change checkpoint model inputs here
- Scale fine-tuning data with the model's target scaler
- Freezing MPNN and FFN layers
- Set up trainer
- Start training
- Test results
- Transfer learning with multicomponent models

Transfer learning (or pretraining) leverages knowledge from a pre-trained model on a related task to enhance performance on a new task. In Chemprop, we can use pre-trained model checkpoints to initialize a new model and freeze components of the new model during training, as demonstrated in this notebook.



```
[1]: # Install chemprop from GitHub if running in Google Colab
import os

if os.getenv("COLAB_RELEASE_TAG"):
    try:
        import chemprop
    except ImportError:
        !git clone https://github.com/chemprop/chemprop.git
        %cd chemprop
        !pip install .
        %cd examples
```

```
[2]: import pandas as pd
      from pathlib import Path

      from lightning import pytorch as pl
      from sklearn.preprocessing import StandardScaler
      import torch

      from chemprop import data, featurizers, models

[3]: chemprop_dir = Path.cwd().parent
      input_path = chemprop_dir / "tests" / "data" / "regression" / "mol" / "mol.csv"
      num_workers = 0 # number of workers for dataloader. 0 means using main process
      smiles_column = 'smiles' # name of the column containing SMILES strings
      target_columns = ['lipo'] # list of names of the columns containing targets
```

## Load data

```
[4]: df_input = pd.read_csv(input_path)
      df_input
```

```
[4]:
```

|     | smiles                                                          | lipo  |
|-----|-----------------------------------------------------------------|-------|
| 0   | <chem>Cn1c(CN2CCN(CC2)c3ccc(Cl)cc3)nc4ccccc14</chem>            | 3.54  |
| 1   | <chem>COc1cc(OC)c(cc1NC(=O)CSCC(=O)O)S(=O)(=O)N2C(C)...</chem>  | -1.18 |
| 2   | <chem>COC(=O)[C@@H](N1CCc2sccc2C1)c3ccccc3Cl</chem>             | 3.69  |
| 3   | <chem>OC[C@H](O)CN1C(=O)C(Cc2ccccc12)NC(=O)c3cc4cc(C...</chem>  | 3.37  |
| 4   | <chem>Cc1cccc(C[C@H](NC(=O)c2cc(nn2C)C(C)(C)C)C(=O)N...</chem>  | 3.10  |
| ... | ...                                                             | ...   |
| 95  | <chem>CC(C)N(CCCNC(=O)Nc1ccc(cc1)C(C)(C)C)C[C@H]2O[C...</chem>  | 2.20  |
| 96  | <chem>CCN(CC)CCCCNc1ncc2CN(C(=O)N(Cc3ccccc(NC(=O)C=C)...</chem> | 2.04  |
| 97  | <chem>CCSc1c(Cc2ccccc2C(F)(F)F)sc3N(CC(C)C)C(=O)N(C)...</chem>  | 4.49  |
| 98  | <chem>COc1ccc(Cc2c(N)n[nH]c2N)cc1</chem>                        | 0.20  |
| 99  | <chem>CCN(CCN(C)C)S(=O)(=O)c1ccc(cc1)c2cnc(N)c(n2)C(...</chem>  | 2.00  |

100 rows × 2 columns

## Get SMILES and targets

```
[5]: smis = df_input.loc[:, smiles_column].values
      ys = df_input.loc[:, target_columns].values
```

```
[6]: smis[:5] # show first 5 SMILES strings
```

```
[6]: array(['Cn1c(CN2CCN(CC2)c3ccc(Cl)cc3)nc4cccc14',
          'COc1cc(OC)c(cc1NC(=O)CSCC(=O)O)S(=O)(=O)N2C(C)CCc3cccc23',
          'COC(=O)[C@@H](N1CCc2sc2C1)c3cccc3Cl',
          'OC[C@H](O)CN1C(=O)C(Cc2cccc12)NC(=O)c3cc4cc(Cl)sc4[nH]3',
          'Cc1cccc(C[C@H](NC(=O)c2cc(nn2C)C(C)(C)C)C(=O)NCC#N)c1'],
          dtype=object)
```

```
[7]: ys[:5] # show first 5 targets
```

```
[7]: array([[ 3.54],
          [-1.18],
          [ 3.69],
          [ 3.37],
          [ 3.1 ]])
```

## Get molecule datapoints

```
[8]: all_data = [data.MoleculeDatapoint.from_smi(smi, y) for smi, y in zip(smi,
```

## Perform data splitting for training, validation, and testing

```
[9]: # available split types
     list(data.SplitType.keys())
```

```
[9]: ['SCAFFOLD_BALANCED',
      'RANDOM_WITH_REPEATED_SMILES',
      'RANDOM',
      'KENNARD_STONE',
      'KMEANS']
```

```
[10]: mols = [d.mol for d in all_data] # RDkit Mol objects are use for structu
      train_indices, val_indices, test_indices = data.make_split_indices(mols,
      train_data, val_data, test_data = data.split_data_by_indices(
          all_data, train_indices, val_indices, test_indices
      )
```

The return type of make\_split\_indices has changed in v2.1 - see help(make\_

Both message-passing neural networks (MPNNs) and multi-component MPNNs can have their weights initialized from a checkpoint file.

```
[11]: chemprop_dir = Path.cwd().parent
      checkpoint_path = chemprop_dir / "tests" / "data" / "example_model_v2_reg
      # If the checkpoint file is generated using the trainin
```

```
[12]: mpnn_cls = models.MPNN
```

```
[13]: mpnn = mpnn_cls.load_from_file(checkpoint_path)
      mpnn
```

```
[13]: MPNN(
  (message_passing): BondMessagePassing(
    (W_i): Linear(in_features=86, out_features=300, bias=False)
    (W_h): Linear(in_features=300, out_features=300, bias=False)
    (W_o): Linear(in_features=372, out_features=300, bias=True)
    (dropout): Dropout(p=0.0, inplace=False)
    (tau): ReLU()
    (V_d_transform): Identity()
    (graph_transform): GraphTransform(
      (V_transform): Identity()
      (E_transform): Identity()
    )
  )
  (agg): MeanAggregation()
  (bn): BatchNorm1d(300, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (predictor): RegressionFFN(
    (ffn): MLP(
      (0): Sequential(
        (0): Linear(in_features=300, out_features=300, bias=True)
      )
      (1): Sequential(
        (0): ReLU()
        (1): Dropout(p=0.0, inplace=False)
        (2): Linear(in_features=300, out_features=1, bias=True)
      )
    )
  )
  (criterion): MSE(task_weights=[[1.0]])
  (output_transform): UnscaleTransform()
)
(X_d_transform): Identity()
(metrics): ModuleList(
  (0-1): 2 x MSE(task_weights=[[1.0]])
)
)
```

If the pre-trained model was a regression model, it probably was trained on a scaled dataset. The scaler is saved as part of the model and used during prediction. For further training, we need to scale the fine-tuning data with the same target scaler.

```
[14]: pretraining_scaler = StandardScaler()
      pretraining_scaler.mean_ = mpnn.predictor.output_transform.mean.numpy()
      pretraining_scaler.scale_ = mpnn.predictor.output_transform.scale.numpy()
```

# Get MoleculeDataset

```
[15]: featurizer = featurizers.SimpleMoleculeMolGraphFeaturizer()

train_dset = data.MoleculeDataset(train_data[0], featurizer)
train_dset.normalize_targets(pretraining_scaler)

val_dset = data.MoleculeDataset(val_data[0], featurizer)
val_dset.normalize_targets(pretraining_scaler)

test_dset = data.MoleculeDataset(test_data[0], featurizer)
```

# Get DataLoader

```
[16]: train_loader = data.build_dataloader(train_dset, num_workers=num_workers)
val_loader = data.build_dataloader(val_dset, num_workers=num_workers, shuffle=False)
test_loader = data.build_dataloader(test_dset, num_workers=num_workers, shuffle=False)
```

Certain layers of a pre-trained model can be kept unchanged during further training on a new task.

# Freezing the MPNN

```
[17]: mpnn.message_passing.apply(lambda module: module.requires_grad_(False))
mpnn.message_passing.eval()
mpnn.bn.apply(lambda module: module.requires_grad_(False))
mpnn.bn.eval() # Set batch norm layers to eval mode to freeze running mean and variance
```

```
[17]: BatchNorm1d(300, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
```

# Freezing FFN layers

```
[18]: frzn_ffn_layers = 1 # the number of consecutive FFN layers to freeze.
```

```
[19]: for idx in range(frzn_ffn_layers):
    mpnn.predictor.ffn[idx].requires_grad_(False)
    mpnn.predictor.ffn[idx + 1].eval()
```

```
[20]: trainer = pl.Trainer(
    logger=False,
    enable_checkpointing=True, # Use `True` if you want to save model che
    enable_progress_bar=True,
    accelerator="auto",
    devices=1,
    max_epochs=20, # number of epochs to train for
)
```

💡 Tip: For seamless cloud uploads and versioning, try installing [litmodelhub]  
 GPU available: False, used: False  
 TPU available: False, using: 0 TPU cores

```
[21]: trainer.fit(mpnn, train_loader, val_loader)
```

/home/knathan/anaconda3/envs/chemprop/lib/python3.11/site-packages/lightn:  
 Loading `train\_dataloader` to estimate number of stepping batches.  
 /home/knathan/anaconda3/envs/chemprop/lib/python3.11/site-packages/lightn:

|   | Name            | Type               | Params | Mode  | FLOPs |
|---|-----------------|--------------------|--------|-------|-------|
| 0 | message_passing | BondMessagePassing | 227 K  | eval  | 0     |
| 1 | agg             | MeanAggregation    | 0      | train | 0     |
| 2 | bn              | BatchNorm1d        | 600    | eval  | 0     |
| 3 | predictor       | RegressionFFN      | 90.6 K | train | 0     |
| 4 | X_d_transform   | Identity           | 0      | train | 0     |
| 5 | metrics         | ModuleList         | 0      | train | 0     |

Trainable params: 301

Non-trainable params: 318 K

Total params: 318 K

Total estimated model params size (MB): 1

Modules in train mode: 11

Modules in eval mode: 15

Total FLOPs: 0

/home/knathan/anaconda3/envs/chemprop/lib/python3.11/site-packages/lightn:  
 tor.py:434: The 'val\_dataloader' does not have many workers which may be  
 value of the `num\_workers` argument` to `num\_workers=11` in the `DataLoa

/home/knathan/anaconda3/envs/chemprop/lib/python3.11/site-packages/lightn:  
 15 module(s) in eval mode at the start of training. This may lead to unex  
 is intentional, you can ignore this warning.

`Trainer.fit` stopped: `max\_epochs=20` reached.

```
[22]: results = trainer.test(mpnn, test_loader, weights_only=False) # weights_
```

| Test metric | DataLoader 0       |
|-------------|--------------------|
| test/mse    | 0.9767233729362488 |

 main ▼

/home/knathan/anaconda3/envs/chemprop/lib/python3.11/site-packages/lightn:

Multi-component MPNN models have individual MPNN blocks for each molecule it parses in one input. These MPNN modules can be independently frozen for transfer learning.

## Change data inputs here

```
[23]: chemprop_dir = Path.cwd().parent
checkpoint_path = chemprop_dir / "tests" / "data" / "example_model_v2_reg"
```

## Change checkpoint model inputs here

```
[24]: mpnn_cls = models.MulticomponentMPNN
mcmpnn = mpnn_cls.load_from_checkpoint(checkpoint_path)
mcmpnn
```

```
[24]: MulticomponentMPNN(
  (message_passing): MulticomponentMessagePassing(
    (blocks): ModuleList(
      (0-1): 2 x BondMessagePassing(
        (W_i): Linear(in_features=86, out_features=300, bias=False)
        (W_h): Linear(in_features=300, out_features=300, bias=False)
        (W_o): Linear(in_features=372, out_features=300, bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
        (tau): ReLU()
        (V_d_transform): Identity()
        (graph_transform): GraphTransform(
          (V_transform): Identity()
          (E_transform): Identity()
        )
      )
    )
  )
  (agg): MeanAggregation()
  (bn): BatchNorm1d(600, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (predictor): RegressionFFN(
    (ffn): MLP(
      (0): Sequential(
        (0): Linear(in_features=600, out_features=300, bias=True)
      )
      (1): Sequential(
        (0): ReLU()
        (1): Dropout(p=0.0, inplace=False)
        (2): Linear(in_features=300, out_features=1, bias=True)
      )
    )
  )
  (criterion): MSE(task_weights=[[1.0]])
  (output_transform): UnscaleTransform()
)
(X_d_transform): Identity()
(metrics): ModuleList(
  (0-1): 2 x MSE(task_weights=[[1.0]])
)
)
```

```
[25]: blocks_to_freeze = [0, 1] # a list of indices of the individual MPNN blo
```

```
[26]: mcmpnn = mpnn_cls.load_from_checkpoint(checkpoint_path)
      for i in blocks_to_freeze:
          mp_block = mcmpnn.message_passing.blocks[i]
          mp_block.apply(lambda module: module.requires_grad_(False))
          mp_block.eval()
      mcmpnn.bn.apply(lambda module: module.requires_grad_(False))
      mcmpnn.bn.eval()
```

```
[26]: BatchNorm1d(600, eps=1e-05, momentum=0.1, affine=True, track_running_stats
```